

DESENVOLVIMENTO DE INTERFACES MODERNAS

Apostila Completa de CustomTkinter com Exemplos e Elementos Visuais

Evolução do Tkinter Clássico para Python

Temas Escuros/Claros, Bordas Arredondadas e Widgets Customizados

2026

1. O que é o CustomTkinter?

O **CustomTkinter** é uma extensão avançada da biblioteca Tkinter tradicional. Desenvolvida para resolver as limitações visuais datadas do kit original, esta biblioteca introduz componentes altamente customizáveis com renderização moderna, cantos arredondados, gradientes suaves e suporte nativo automático a **Modo Escuro (Dark Mode)** e **Modo Claro (Light Mode)**.

Principais Vantagens: Ao contrário do Tkinter clássico, que herda a aparência nativa (muitas vezes obsoleta) do sistema operativo, o CustomTkinter desenha os seus próprios widgets diretamente na tela através do Canvas, garantindo consistência visual idêntica em Windows, macOS e Linux.

Instalação do Pacote

Por ser uma biblioteca externa, sua instalação é feita de forma simples através do gerenciador de pacotes do Python:

```
pip install customtkinter
```

O Primeiro Código: Janela Moderna

```
import customtkinter as ctk

# Configuração global de aparência
ctk.set_appearance_mode("System") # Opções: "System", "Dark", "Light"
ctk.set_default_color_theme("blue") # Opções: "blue", "green", "dark-blue"

root = ctk.CTk() # Em vez de tk.Tk()
root.title("Minha GUI CustomTkinter")
root.geometry("400x300")

root.mainloop()
```

2. Desenho Arquitetural da Janela e Temas

O CustomTkinter gerencia os widgets encadeando propriedades de cantos (`corner_radius`) e cores de estados dinâmicos (passagem do mouse/clique).

Desenho do Comportamento de Cor Dinâmica (Hover)

Modo Escuro (Dark)	Modo Claro (Light)
<pre>fg_color: "#1f538d" text_color: "#ffffff" [Passar Rato (Hover)] hover_color: "#14375e"</pre>	<pre>fg_color: "#3b8ed0" text_color: "#000000" [Passar Rato (Hover)] hover_color: "#2c74b3"</pre>

3. Catálogo e Sintaxe de Widgets Avançados

Abaixo estão detalhados os principais componentes substituídos pelo CustomTkinter, suas sintaxes e as propriedades exclusivas de customização.

A) Botão customizado: CTkButton

Permite controle total sobre o raio de arredondamento e cores independentes para quando o mouse passa sobre o elemento.

```
btn = ctk.CTkButton(
    master=root,
    text="Clique Aqui",
    command=minha_funcao,
    corner_radius=10,          # Controla o arredondamento dos cantos
    fg_color="#2ecc71",        # Cor de fundo estável
    hover_color="#27ae60",     # Cor ao passar o mouse
    text_color="white"
)
btn.pack(pady=10)
```

B) Campo de Entrada: CTkEntry

Adiciona suporte nativo a placeholders (textos de dica interna que desaparecem ao digitar) de forma simplificada.

```

campo = ctk.CTkEntry(
    master=root,
    placeholder_text="Digite o seu e-mail...",
    width=250,
    height=40,
    border_width=2,
    corner_radius=8
)
campo.pack(pady=10)

```

C) Controle de Seleção Alternável: CTkSwitch

Um elemento de interruptor moderno estilo iOS/Android que substitui com elegância os tradicionais checkboxes.

```

switch_var = ctk.StringVar(value="on")

switch = ctk.CTkSwitch(
    master=root,
    text="Ativar Notificações",
    variable=switch_var,
    onvalue="on",
    offvalue="off"
)
switch.pack(pady=10)

```

D) Barra de Progresso Suave: CTkProgressBar

Utilizada para exibir feedbacks visuais de carregamento ou download.

```

progresso = ctk.CTkProgressBar(master=root, width=300)
progresso.pack(pady=10)
progresso.set(0.65) # Define a barra em 65%

```

4. Gerenciamento de Telas Complexas e Abas

O CustomTkinter estende o conceito de Frames com os objetos CTkFrame e introduz o espetacular CTkTabview, facilitando a criação de painéis segmentados sem precisar reescrever lógicas de ocultação de janelas.

Representação Visual do CTkTabview

```
+=====+
| [ Tab 1: Perfil ] [ Tab 2: Configurações ] |
+=====+
|
| Nome: [ _____ ]
| Idade: [ ____ ]
|
+-----+
```

```
import customtkinter as ctk

root = ctk.CTk()
root.geometry("450x350")
root.title("Uso de Abas Avançadas")

# Criando o painel de abas
abas = ctk.CTkTabview(root, width=400, height=300)
abas.pack(padx=20, pady=20)

# Adicionando abas filhas
tab_perfil = abas.add("Perfil")
tab_config = abas.add("Configurações")

# Adicionando conteúdo dentro da primeira aba
lbl = ctk.CTkLabel(tab_perfil, text="Dados do Usuário", font=("Arial", 14, "bold"))
lbl.pack(pady=10)

txt_nome = ctk.CTkEntry(tab_perfil, placeholder_text="Nome do Perfil")
txt_nome.pack(pady=5)

# Conteúdo da segunda aba
switch_modos = ctk.CTkSwitch(tab_config, text="Modo Escuro Forçado")
switch_modos.pack(pady=20)

root.mainloop()
```

5. Projeto Prático Final: Painel de Controle (Dashboard)

O código estruturado abaixo compõe uma tela de login moderna com transição simulada de dados utilizando as melhores práticas do CustomTkinter.

```

import customtkinter as ctk

class AppDashboard(ctk.CTk):
    def __init__(self):
        super().__init__()

        self.title("Painel Administrativo Moderno")
        self.geometry("500x400")
        self.resizable(False, False)

        # Configuração do Layout de Fundo do Card
        self.card = ctk.CTkFrame(self, width=400, height=320, corner_radius=15)
        self.card.place(relx=0.5, rely=0.5, anchor="center")

        # Elementos internos do Card
        self.titulo = ctk.CTkLabel(self.card, text="Acesso ao Sistema", font=("Helvetica",
20, "bold"))
        self.titulo.pack(pady=(30, 20))

        self.input_user = ctk.CTkEntry(self.card, placeholder_text="Nome de Usuário",
width=280, height=35)
        self.input_user.pack(pady=10)

        self.input_pass = ctk.CTkEntry(self.card, placeholder_text="Senha de Acesso",
show="*", width=280, height=35)
        self.input_pass.pack(pady=10)

        # Elemento Switch de Seleção de Tema Rápido
        self.switch_tema = ctk.CTkSwitch(self.card, text="Alternar Tema Escuro",
command=self.mudar_tema)
        self.switch_tema.pack(pady=15)

        self.btn_login = ctk.CTkButton(self.card, text="Autenticar", width=280, height=40,
corner_radius=8, font=("Arial", 12, "bold"), command=self.autenticar)
        self.btn_login.pack(pady=(10, 20))

    def mudar_tema(self):
        if ctk.get_appearance_mode() == "Dark":
            ctk.set_appearance_mode("Light")
        else:
            ctk.set_appearance_mode("Dark")

    def autenticar(self):
        usuario = self.input_user.get()
        if usuario:
            self.titulo.configure(text=f"Bem-vindo, {usuario}!", text_color="#2ecc71")
        else:
            self.titulo.configure(text="Usuário Inválido", text_color="#e74c3c")

```

```
if __name__ == "__main__":  
    app = AppDashboard()  
    app.mainloop()
```